

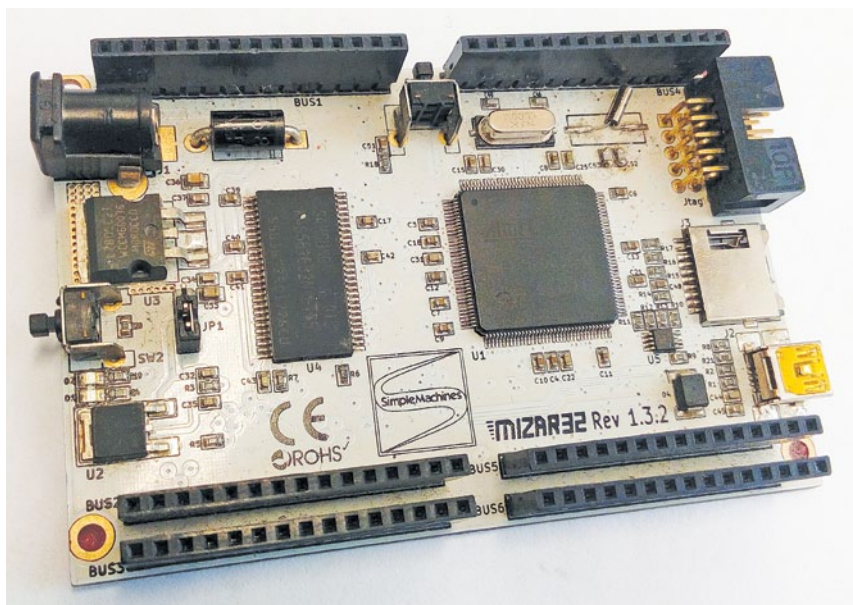


Fig. 3: Mizar32 development board

interactive and incremental way. The code supplies various tools to aid in native Lisp programming (like an onboard vi-clone text editor and XMODEM implementation to share files).

4. PicoLisp is a very extensible piece of software. Adding support for newer peripherals or modules is a naturally-smooth process.
5. Code bears a royalty-free, permissive, non-copyleft free software licence. This permits code reuse within a proprietary digital base.

Examples with Mizar32 board

Mizar32 development board (Fig. 3) is a low-cost, stackable multi-board computer designed and produced under Free Hardware Licence by SimpleMachines. It is composed of a base module with Atmel AVR32 AT32UC3A0256 system-on-chip processor running at 66MHz with 32MB of SDRAM, 64kB SRAM, 256kB of flash memory, SD card slot, USB port and JTAG connector. A range of add-on boards can be stacked on its bus connectors to provide a serial port, a character based LCD display, VGA/mouse/keyboard/audio board, Ethernet and so on.

It runs PicoLisp as its standard

software, allowing fast production of custom solutions.

Mizar32 is shipped with PicoLisp interpreter programmed into flash, including support for the optional TCP/IP networking module.

Mizar32 can also be programmed by the end user with different, customised versions of PicoLisp, FreeRTOS or with other stand-alone programs.

Once we have PicoLisp running on the board, you can access all MCU peripherals from Lisp. There are three Mizar32 variants, namely, A, B and C. These use AT32UC3A0512, AT32UC3A0256 and AT32UC3A0128 chips, respectively. SDRAM interface is the same for all three, which is 32MB. The difference is in flash size. Otherwise, the board layout, memory layout is the same.

Let us now see two examples:

Toggling LED. A simple PicoLisp program (led.l) is listed below that toggles an LED using a user button on Mizar32 board. Quoted symbols PB_29 and PX_16 are PicoLisp symbols that correspond to pin 29 on Port B and pin 16 on Port X, respectively. Parsing of these values is done in the generic PIO Lisp module (via hardware-abstraction layer). Tran-

sient symbols `*pio-output*` and `*pio-input*` are used to set directions to the port pins. A simple infinite loop reads the button on the input pin and toggles the LED if pressed.

```
# A sample for user-buttons.
# Declare pins
(setq led 'PB_29 button 'PX_16)
# A simple delay function
(de delay (t)
  (tmr-delay 0 t) )
# Make sure the LED starts in
# the "off" position and enable
# input/output pins
(de init-pins ()
  (pio-pin-sethigh led)
  (pio-pin-setdir *pio-output* led)
  (pio-pin-setdir *pio-input* button) )
# And now, the main loop
(de prog-loop ()
  (init-pins)
  (loop
    (if (= 0 (pio-pin-getval button))
      (pio-pin-setlow led)
      (delay 100000)
      (pio-pin-sethigh led)
      (delay 100000) ) ) )
(prog-loop)
```

Reaction timer. This program (timer1.l) measures the time taken to press a key on the keyboard after the program is run. Time will be displayed on the terminal. Try it a few times and then see results. Also compare results with others who have tried this before you:

```
# A reaction timer in picolisp
(de reaction-timer ()
  (prnl "Welcome to the reaction timer.
  When I say Go!, enter a character.")
  (prnl "Press q [Enter] to quit.")
  (setq
    timer *tmr-sys-timer*
    answer "" )
  (until (=T answer)
    (println "Ready?")
    # Wait for a random time
    from 2 to 5 seconds
    (tmr-delay timer (+ 2000000 (rand 1
      3000000)))
    (println "Go!")
    (setq
      start-time (tmr-read timer)
      answer (read) # wait for them to
```